RabbitMQ

JavaScript

Messaging patterns with amqplib

by Bhuvnesh Verma

https://www.rabbitmq.com/tutorials

Contents

1. Getting Started

Short description: RabbitMQ is a message broker. It accepts, stores, and forwards messages between applications. Think of it as a post office: you drop mail into a post box, and the postal service delivers it.

Theory: Applications do not talk to each other directly. A producer hands a message to the broker, the broker stores it in a queue, and a consumer picks it up when it is ready. This decouples the two sides in both time and space.

How it works: A producer publishes to an exchange. The exchange applies its routing rules and places the message into one or more queues. A consumer subscribes to a queue and receives messages as they arrive.

Use case example: An online store accepts an order and must send a confirmation email, update inventory, and notify shipping. Publishing one order event lets three independent services react without the store waiting for any of them.

Key Points

- **Producer** sends messages
- **Queue** stores messages, like a post box
- **Consumer** receives messages
- **Exchange** routes messages to queues
- The Node.js client library is `amqplib`

Notes

- Every example assumes a broker running on `localhost` at port 5672
- `amqplib` is promise based, so every call happens inside an async function
- A channel, not a connection, is the unit you publish and consume on

1.1 Installation and connection

```
npm install amqplib
```

```
const amqp = require('amqplib');

async function main() {
    // 1. Open a TCP connection to the broker
    const connection = await amqp.connect('amqp://localhost');

    // 2. Open a channel: all AMQP operations happen here
    const channel = await connection.createChannel();

    // ... declare queues, publish, consume ...

    // 3. Always close when finished
    await channel.close();
    await connection.close();
}

main().catch(console.error);
```

2. Hello World

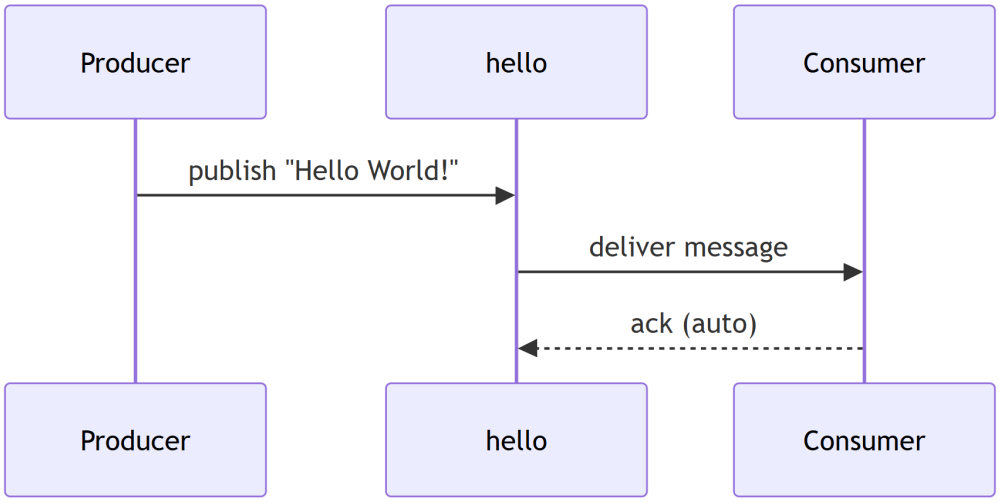
Short description: The simplest pattern. One producer sends one message to one queue, and one consumer receives it.

Theory: Point to point messaging. Producer → Queue → Consumer. No exchange is used, so messages go through the default exchange.

How it works: The producer declares a queue and sends a message to it. The consumer declares the same queue and consumes from it. Declaring is idempotent, so whichever side starts first creates the queue.

Use case example: A simple notification service where an order placement service sends a "new order" message to a queue, and an inventory service picks it up to update stock.

2.1 Architecture diagram



Key Points

- The queue must be declared before sending or receiving
- An empty exchange name (`''`) uses the **default exchange**
- `noAck: true` means the broker treats a message as delivered immediately

Notes

- Run the producer once; the consumer stays running
- Messages are **not persistent** by default and are lost if RabbitMQ restarts

2.2 Code examples

Producer (`send.js`)

```
const amqp = require('amqplib');

async function send() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const queue = 'hello';
    await channel.assertQueue(queue, { durable: false });

    const msg = 'Hello World!';
    channel.sendToQueue(queue, Buffer.from(msg));
    console.log(" [x] Sent '%s'", msg);

    setTimeout(() => { connection.close(); }, 500);
}
send();
```

Consumer (`receive.js`)

```
const amqp = require('amqplib');

async function receive() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const queue = 'hello';
    await channel.assertQueue(queue, { durable: false });

    console.log(" [*] Waiting for messages. To exit press CTRL+C");

    channel.consume(queue, (msg) => {
        console.log(" [x] Received '%s'", msg.content.toString());
    }, { noAck: true });
}
receive();
```

3. Work Queues

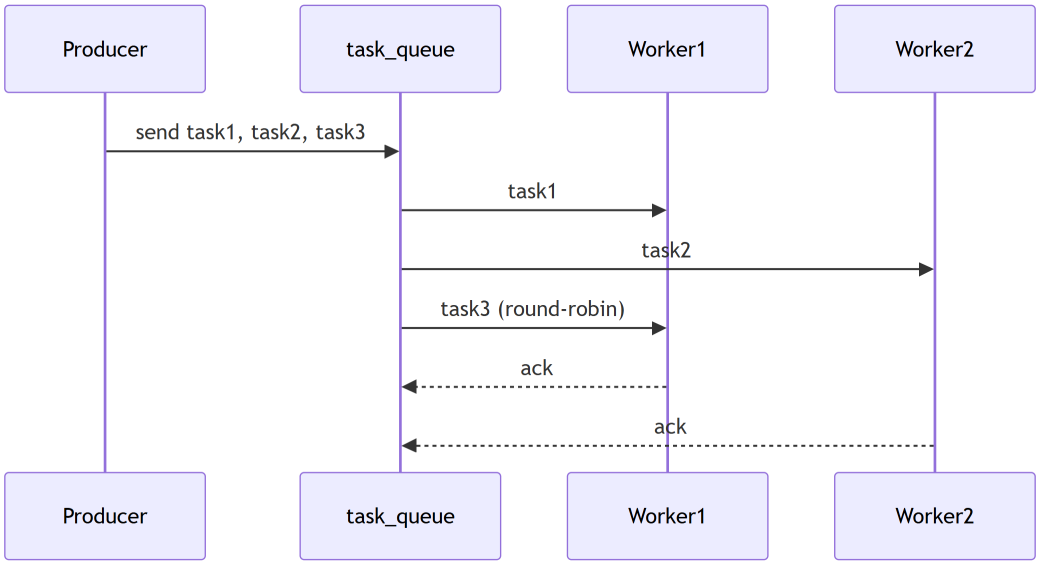
Short description: Distributes time consuming tasks among multiple workers. This is the competing consumers pattern.

Theory: Avoid doing resource intensive work inside a request. Schedule it to be done later by a pool of workers that share one queue.

How it works: The producer sends tasks to a queue. Multiple workers consume from the same queue, and tasks are handed out **round robin**. Each worker acknowledges a task only once it has finished, so nothing is lost if a worker dies mid job.

Use case example: A video processing service receives upload requests. Each video is transcoded by a worker. Workers run as multiple instances and tasks are distributed evenly. If a worker crashes, its task is re-queued for another worker.

3.1 Architecture diagram



Key Points

- `durable: true` makes the queue survive a broker restart
- `persistent: true` makes the message survive a broker restart
- `prefetch(1)` gives fair dispatch, so no worker is handed a backlog
- **Always acknowledge**, otherwise messages can be lost

Notes

- Round robin distribution gives each worker one message at a time
- If a worker dies without acknowledging, the message is re-queued
- Durability and persistence are separate settings, and you usually want both

3.2 Code examples

Producer (`new_task.js`)

```
const amqp = require('amqplib');

async function send() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const queue = 'task_queue';
    await channel.assertQueue(queue, { durable: true });

    const msg = process.argv.slice(2).join(' ') || "Hello World!";
    channel.sendToQueue(queue, Buffer.from(msg), { persistent: true });
    console.log(" [x] Sent '%s'", msg);

    setTimeout(() => { connection.close(); }, 500);
}
send();
```

Worker (`worker.js`)

```
const amqp = require('amqplib');

async function receive() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const queue = 'task_queue';
    await channel.assertQueue(queue, { durable: true });

    channel.prefetch(1);
    console.log(" [*] Waiting for messages. To exit press CTRL+C");

    channel.consume(queue, (msg) => {
        const secs = msg.content.toString().split('.').length - 1;
        console.log(" [x] Received %s", msg.content.toString());
        setTimeout(() => {
            console.log(" [x] Done");
            channel.ack(msg);
        }, secs * 1000);
    }, { noAck: false });
}
receive();
```

4. Publish and Subscribe

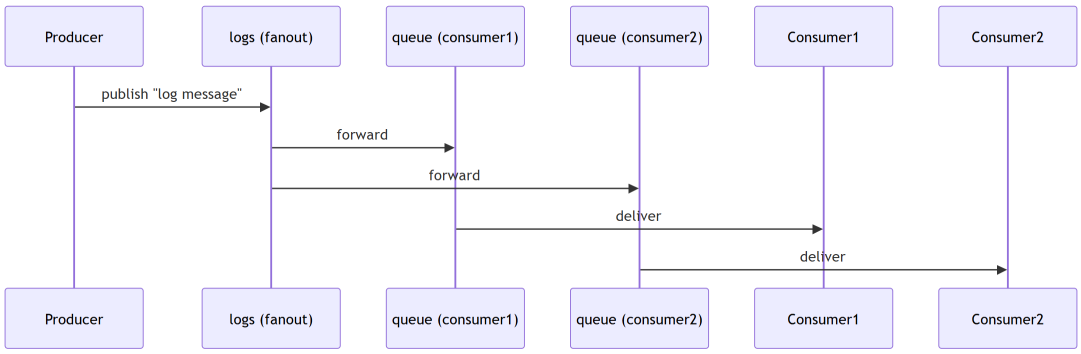
Short description: Broadcasts every message to **multiple consumers** at once.

Theory: The producer sends to a **fanout exchange**. The exchange forwards each message to **all bound queues**, and every consumer owns its own queue.

How it works: The producer declares a fanout exchange. Each consumer creates a temporary, exclusive queue and binds it to that exchange. Every message published to the exchange is copied into all bound queues.

Use case example: A logging system where every log line must reach several sinks at once: a file logger, a monitoring dashboard, and an alerting service. Each sink runs as a separate consumer with its own queue.

4.1 Architecture diagram



Key Points

- A **fanout exchange** ignores routing keys and broadcasts to everything bound
- Temporary queues declared as `''` get a random generated name
- `exclusive: true` deletes the queue when its consumer disconnects
- Each consumer receives **all** messages

Notes

- This differs from Work Queues, where consumers share the messages
- Here each consumer gets its own **copy** of every message
- Messages published while no queue is bound are simply discarded

4.2 Code examples

Producer (`emit_log.js`)

```
const amqp = require('amqplib');

async function send() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const exchange = 'logs';
    await channel.assertExchange(exchange, 'fanout', { durable: false });

    const msg = process.argv.slice(2).join(' ') || "info: Hello World!";
    channel.publish(exchange, '', Buffer.from(msg));
    console.log(" [x] Sent '%s'", msg);

    setTimeout(() => { connection.close(); }, 500);
}
send();
```

Consumer (`receive_logs.js`)

```
const amqp = require('amqplib');

async function receive() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const exchange = 'logs';
    await channel.assertExchange(exchange, 'fanout', { durable: false });

    const q = await channel.assertQueue('', { exclusive: true });
    await channel.bindQueue(q.queue, exchange, '');

    console.log(" [*] Waiting for logs. To exit press CTRL+C");

    channel.consume(q.queue, (msg) => {
        console.log(" [x] %s", msg.content.toString());
    }, { noAck: true });
}
receive();
```

5. Routing

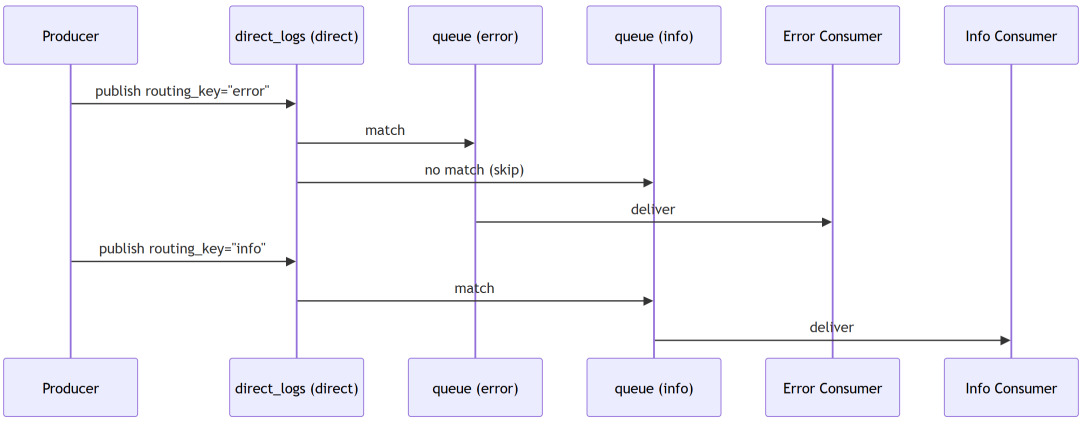
Short description: Selectively receives messages based on **routing keys**, matched exactly.

Theory: Uses a **direct exchange**. A message is routed to every queue whose **binding key exactly matches** the message routing key.

How it works: The producer publishes with a routing key such as “error” or “info”. Consumers bind their queues to the exchange with the specific keys they care about, and the exchange delivers only to the queues that match.

Use case example: A log aggregator where services emit logs at different severity levels. Only “error” logs reach the on call pager, while “info” and “warning” go to database storage. Each consumer binds to the severities it wants.

5.1 Architecture diagram



Key Points

- A **direct exchange** routes by exact match
- Several queues may bind with different routing keys
- One queue may bind with **multiple** routing keys

Notes

- Run several consumers with different severity arguments to see the split
- Example: `node receive_logs_direct.js error warning`
- A message whose key matches nothing is dropped unless you publish with `mandatory`

5.2 Code examples

Producer (`emit_log_direct.js`)

```
const amqp = require('amqplib');

async function send() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const exchange = 'direct_logs';
    await channel.assertExchange(exchange, 'direct', { durable: false });

    const severity = process.argv[2] || 'info';
    const msg = process.argv.slice(3).join(' ') || "Hello World!";
    channel.publish(exchange, severity, Buffer.from(msg));
    console.log(" [x] Sent %s: '%s'", severity, msg);

    setTimeout(() => { connection.close(); }, 500);
}
send();
```

Consumer (`receive_logs_direct.js`)

```
const amqp = require('amqplib');

async function receive() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const exchange = 'direct_logs';
    await channel.assertExchange(exchange, 'direct', { durable: false });

    const q = await channel.assertQueue('', { exclusive: true });

    const severities = process.argv.slice(2) || ['info'];
    for (const severity of severities) {
        await channel.bindQueue(q.queue, exchange, severity);
    }

    console.log(" [*] Waiting for %s. To exit press CTRL+C", severities);

    channel.consume(q.queue, (msg) => {
        console.log(" [x] %s: '%s'", msg.fields.routingKey, msg.content.toString());
    }, { noAck: true });
}
receive();
```

6. Topics

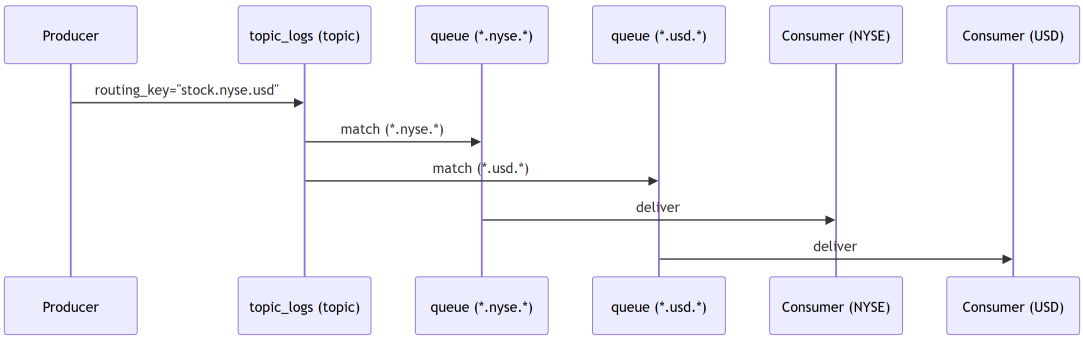
Short description: Receives messages by **pattern matching** on routing keys, using wildcards.

Theory: Uses a **topic exchange**. Routing keys are dot separated words, and consumers bind with patterns built from two wildcards: `*` matches exactly one word, `#` matches zero or more words.

How it works: The producer publishes with a routing key such as `stock.usd.nyse`. A consumer binds a pattern such as `*.usd.#` and receives every message whose key fits that shape.

Use case example: A stock price feed. One client wants everything from the NYSE (`*.nyse.*`), another wants only USD denominated stocks from any exchange (`*.usd.*`), and a third wants the whole energy sector (`energy.#`).

6.1 Architecture diagram



Key Points

- `*` matches one word: `*.stock.*` matches `usd.stock.nyse`
- `#` matches zero or more: `#.error` matches `app.service.error`
- Binding `*.orange.*` matches `quick.orange.rabbit`
- Binding `lazy.#` matches both `lazy.orange.rabbit` and `lazy`

Notes

- A topic exchange bound with `#` behaves exactly like a fanout exchange
- Bound with a key containing no wildcard, it behaves like a direct exchange
- Very flexible, and the usual choice for complex event filtering

6.2 Code examples

Producer (`emit_log_topic.js`)

```
const amqp = require('amqplib');

async function send() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const exchange = 'topic_logs';
    await channel.assertExchange(exchange, 'topic', { durable: false });

    const routingKey = process.argv[2] || 'anonymous.info';
    const msg = process.argv.slice(3).join(' ') || "Hello World!";
    channel.publish(exchange, routingKey, Buffer.from(msg));
    console.log(" [x] Sent %s: '%s'", routingKey, msg);

    setTimeout(() => { connection.close(); }, 500);
}
send();
```

Consumer (`receive_logs_topic.js`)

```
const amqp = require('amqplib');

async function receive() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const exchange = 'topic_logs';
    await channel.assertExchange(exchange, 'topic', { durable: false });

    const q = await channel.assertQueue('', { exclusive: true });

    const bindingKeys = process.argv.slice(2) || ['#'];
    for (const key of bindingKeys) {
        await channel.bindQueue(q.queue, exchange, key);
    }

    console.log(" [*] Waiting for %s. To exit press CTRL+C", bindingKeys);

    channel.consume(q.queue, (msg) => {
        console.log(" [x] %s: '%s'", msg.fields.routingKey, msg.content.toString());
    }, { noAck: true });
}
receive();
```

7. Remote Procedure Call

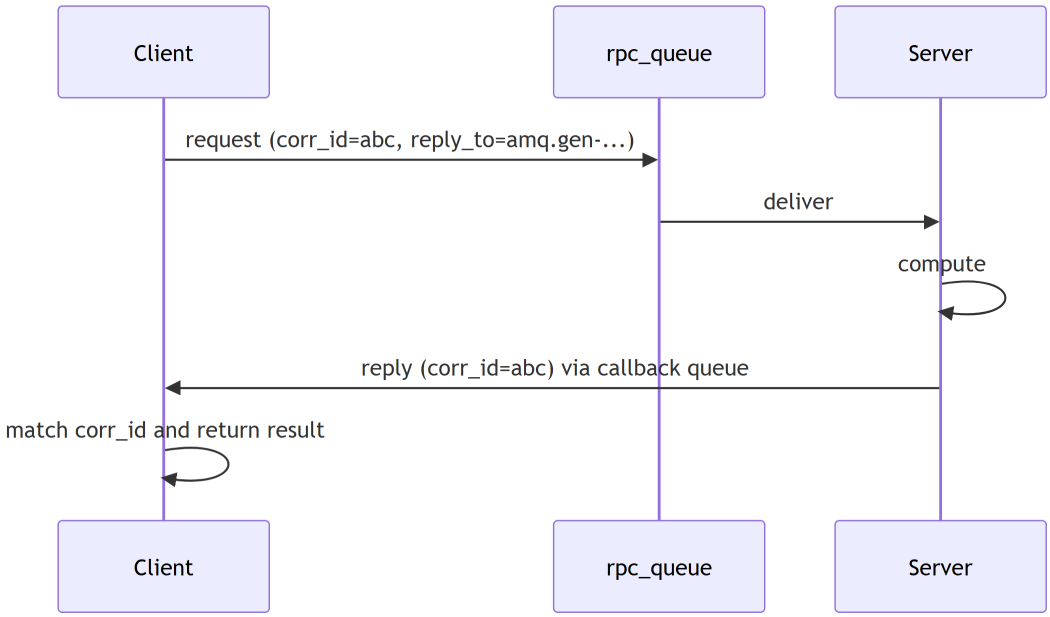
Short description: The client sends a request and waits for a response, much like calling a function that happens to live on another machine.

Theory: The client sends a request carrying a **callback queue** name and a **correlation ID**. The server processes it and publishes the result to that callback queue. The client listens on the callback queue and matches replies by correlation ID.

How it works: The client creates a temporary exclusive queue for replies and sets `replyTo` and `correlationId` on the request. The server consumes the request, computes, and sends the result to `replyTo` with the same correlation ID. The client resolves the matching promise.

Use case example: A web frontend needs a Fibonacci number that is expensive to compute, so the work is offloaded to an RPC server. The frontend sends the input and waits for the result to come back.

7.1 Architecture diagram



Key Points

- The **callback queue** is where the server sends the response
- The **correlation ID** matches a response to its request and prevents mixups
- The client suspends until the reply arrives

Notes

- RPC is synchronous from the caller's point of view, so it adds latency
- Good when you genuinely need a result back, otherwise prefer a plain queue
- Always set a timeout, since a dead server means a promise that never resolves

7.2 Code examples

RPC Server (`rpc_server.js`)

```
const amqp = require('amqplib');

async function fib(n) {
    return n < 2 ? n : await fib(n - 1) + await fib(n - 2);
}

async function startServer() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    const queue = 'rpc_queue';
    await channel.assertQueue(queue, { durable: false });
    channel.prefetch(1);
    console.log(' [x] Awaiting RPC requests');

    channel.consume(queue, async (msg) => {
        const n = parseInt(msg.content.toString());
        console.log(` [.] fib(${n})`);
        const result = await fib(n);
        channel.sendToQueue(msg.properties.replyTo, Buffer.from(result.toString()), {
            correlationId: msg.properties.correlationId
        });
        channel.ack(msg);
    });
}
startServer();
```

RPC Client (`rpc_client.js`)

```
const amqp = require('amqplib');
const { v4: uuidv4 } = require('uuid');

class RpcClient {
    async connect() {
        this.connection = await amqp.connect('amqp://localhost');
        this.channel = await this.connection.createChannel();
        const q = await this.channel.assertQueue('', { exclusive: true });
        this.callbackQueue = q.queue;
    }

    async call(n) {
        const corrId = uuidv4();
        return new Promise((resolve) => {
            this.channel.consume(this.callbackQueue, (msg) => {
                if (msg.properties.correlationId === corrId) {
                    resolve(parseInt(msg.content.toString()));
                }
            }, { noAck: true });

            this.channel.sendToQueue('rpc_queue', Buffer.from(n.toString()), {
                correlationId: corrId,
                replyTo: this.callbackQueue
            });
        });
    }
}

async function run() {
    const client = new RpcClient();
    await client.connect();
    console.log(" [x] Requesting fib(30)");
    console.log(" [.] Got %d", await client.call(30));
}
run();
```

8. Publisher Confirms

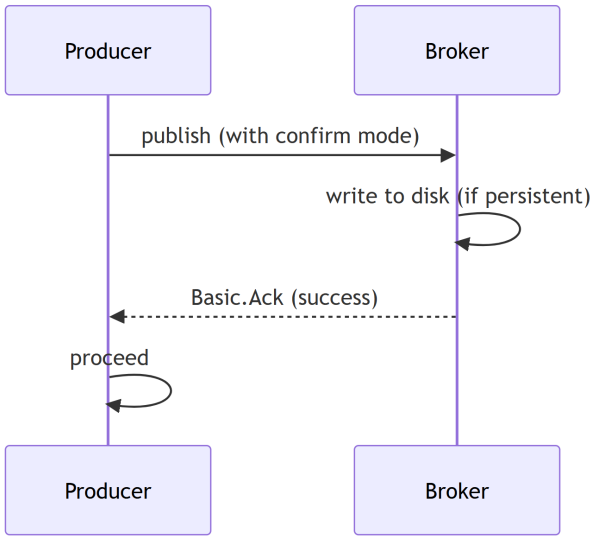
Short description: Guarantees that the broker **actually received** the messages you published.

Theory: The channel is put into confirm mode. Every published message is then either acknowledged or negatively acknowledged by the broker. For persistent messages the confirmation arrives only **after** the message has been written to disk.

How it works: Create the channel with `createConfirmChannel()`. After each publish the broker replies with `Basic.Ack` or `Basic.Nack`. You can wait for these synchronously with `waitForConfirms()` or handle them asynchronously through channel events.

Use case example: An order processing system that cannot lose an order. Each order is published with confirms so the service knows the broker has accepted and persisted it before moving on. If no confirm arrives within a timeout, the service retries.

8.1 Architecture diagram



Key Points

- `createConfirmChannel()` enables confirms
- `Basic.Ack` means the broker took responsibility for the message
- `Basic.Nack` means it did not, for example a routing error
- `waitForConfirms()` blocks until every outstanding message is confirmed

Notes

- Confirms say nothing about consumers, only about the broker
- Pair them with the `mandatory` flag to catch unroutable messages
- Synchronous confirms are slow, so use the async form for high throughput

8.2 Code examples

Synchronous confirms (`confirm_publisher.js`)

```
const amqp = require('amqplib');

async function publish() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createConfirmChannel();  // Confirm channel

    const queue = 'confirm_queue';
    await channel.assertQueue(queue, { durable: true });

    // Publish with a per-message callback
    channel.sendToQueue(queue, Buffer.from('Critical message'), { persistent: true },
        (err, ok) => {
            if (err) {
                console.log(" [x] Message NOT confirmed");
            } else {
                console.log(" [x] Message confirmed");
            }
        });

    await channel.waitForConfirms();
    console.log(" [x] All messages confirmed");

    setTimeout(() => { connection.close(); }, 500);
}
publish();
```

Asynchronous confirms (high throughput)

```
const amqp = require('amqplib');

async function publish() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createConfirmChannel();

    const queue = 'confirm_queue';
    await channel.assertQueue(queue, { durable: true });

    // Listen for confirms instead of waiting on each publish
    channel.on('ack', (msg) => console.log(' [✓] Message acked'));
    channel.on('nack', (msg) => console.log(' [×] Message nacked'));

    for (let i = 0; i < 10; i++) {
        channel.sendToQueue(queue, Buffer.from(`Message ${i}`), { persistent: true });
        console.log(` [x] Sent message ${i}`);
    }

    await channel.waitForConfirms();
    console.log(" [x] All messages confirmed");

    setTimeout(() => { connection.close(); }, 500);
}
publish();
```

9. Queue Types

Short description: RabbitMQ offers three queue types. The type is fixed at declaration time and decides how the queue stores messages and survives a node failure.

Theory:

- A **classic** queue lives on one node
- A **quorum** queue replicates its contents across an odd number of nodes using the Raft consensus algorithm
- A **stream** keeps an append only log that many consumers can read independently, each at its own offset

How it works: You pick the type with the `x-queue-type` argument when declaring the queue. Classic is the default. Quorum trades a little throughput for durability. A stream does not remove a message when it is read, so the same message can be replayed by a new consumer later.

Use case example: Payment confirmations go to a quorum queue because losing one is unacceptable. Thumbnail generation jobs go to a classic queue because they are cheap to retry. An event feed that analytics, billing, and audit all read goes to a stream.

Type	Stores	Pick it when
Classic	One node, optionally on disk	Throughput matters more than surviving a node loss
Quorum	Replicated by Raft, always on disk	The message must not be lost
Stream	Append only log, replayable	Many consumers read the same history

Key Points

- The type cannot be changed later, you must delete and redeclare the queue
- Quorum queues need a majority of nodes alive, so use 3 or 5, never an even number
- A stream keeps messages after delivery, bounded by `x-max-length-bytes` or `x-max-age`

Notes

- Quorum queues do not support priorities or the `exclusive` flag
- Mirrored classic queues are deprecated, quorum queues replace them
- Redeclaring an existing queue with different arguments raises `PRECONDITION_FAILED` and kills the channel

9.1 Code examples

Declaring each type (`queue_types.js`)

```
const amqp = require('amqplib');

async function declare() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    // Classic: the default, single node.
    await channel.assertQueue('jobs.classic', { durable: true });

    // Quorum: replicated across the cluster, always durable.
    await channel.assertQueue('payments.quorum', {
        durable: true,
        arguments: {
            'x-queue-type': 'quorum',
            'x-delivery-limit': 5   // give up after 5 redeliveries
        }
    });

    // Stream: append only log, capped by size and age.
    await channel.assertQueue('events.stream', {
        durable: true,
        arguments: {
            'x-queue-type': 'stream',
            'x-max-length-bytes': 2_000_000_000,
            'x-max-age': '7D'
        }
    });

    console.log(' [x] Declared classic, quorum and stream queues');
    await connection.close();
}
declare();
```

Reading a stream from an offset (`stream_consumer.js`)

```
const amqp = require('amqplib');

async function consume() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    // Streams require a prefetch, the broker refuses the consumer without one.
    await channel.prefetch(100);

    await channel.consume('events.stream', (msg) => {
        console.log(' [x] offset %s: %s',
            msg.properties.headers['x-stream-offset'],
            msg.content.toString());
        channel.ack(msg);
    }, {
        noAck: false,
        arguments: { 'x-stream-offset': 'first' }  // or 'last', 'next', a number
    });
}
consume();
```

10. Message Structure and Message Properties

Short description: Every message carries a body of raw bytes plus a set of properties and headers that travel with it.

Theory: An AMQP message has three parts.

- The **body** is an opaque byte array, RabbitMQ never looks inside it
- The **properties** are a fixed set of fields defined by the protocol, such as `contentType` and `correlationId`
- The **headers** are a free form map you fill with anything your application needs

How it works: In amqplib the body is always a `Buffer`, so objects must be serialised before publishing and parsed after receiving. Properties are passed as the third argument to `publish` and read back on `msg.properties`. Delivery metadata such as the routing key and the delivery tag arrives separately on `msg.fields`.

Use case example: A service publishes JSON with `contentType: 'application/json'`, a `messageId` for deduplication, and a `traceId` header so the request can be followed across every service that touches it.

Property	What it is for
`persistent`	Write the message to disk so a broker restart keeps it
`contentType`	How the consumer should parse the body
`correlationId`	Ties an RPC reply back to its request
`replyTo`	Queue the reply should be sent to
`messageId`	Unique id, used for deduplication and logging
`expiration`	Per message TTL in milliseconds, as a string
`priority`	Rank inside a priority queue
`headers`	Your own key value map, also used by header exchanges

Key Points

- The body is bytes, the broker never parses or validates it
- `persistent: true` plus a durable queue is what survives a restart
- `expiration` is a string of milliseconds, not a number

Notes

- Keep the body small, put routing information in headers instead
- `priority` needs `x-max-priority` set on the queue
- `msg.fields.redelivered` tells you this message was delivered before

10.1 Code examples

Publishing with properties (`publish_properties.js`)

```
const amqp = require('amqplib');
const crypto = require('crypto');

async function send() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    await channel.assertQueue('orders', { durable: true });

    const payload = { orderId: 42, total: 199.5, currency: 'EUR' };

    channel.sendToQueue('orders', Buffer.from(JSON.stringify(payload)), {
        persistent: true,                    // survive a broker restart
        contentType: 'application/json',
        contentEncoding: 'utf-8',
        messageId: crypto.randomUUID(),
        correlationId: 'checkout-42',
        timestamp: Date.now(),
        appId: 'checkout-service',
        type: 'order.created',
        expiration: '60000',                 // milliseconds, as a string
        headers: {
            traceId: 'abc-123',
            attempt: 1,
            source: 'web'
        }
    });

    console.log(' [x] Sent order with properties');
    setTimeout(() => connection.close(), 500);
}
send();
```

Reading them back (`read_properties.js`)

```
const amqp = require('amqplib');

async function receive() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    await channel.assertQueue('orders', { durable: true });
    await channel.prefetch(1);

    channel.consume('orders', (msg) => {
        const { properties: p, fields: f, content } = msg;

        console.log(' [x] routingKey  :', f.routingKey);
        console.log('     redelivered :', f.redelivered);
        console.log('     messageId   :', p.messageId);
        console.log('     type        :', p.type);
        console.log('     traceId     :', p.headers.traceId);

        const body = p.contentType === 'application/json'
            ? JSON.parse(content.toString())
            : content.toString();

        console.log('     body        :', body);
        channel.ack(msg);
    }, { noAck: false });
}
receive();
```

11. Dead Letter Exchange (DLX) and TTL

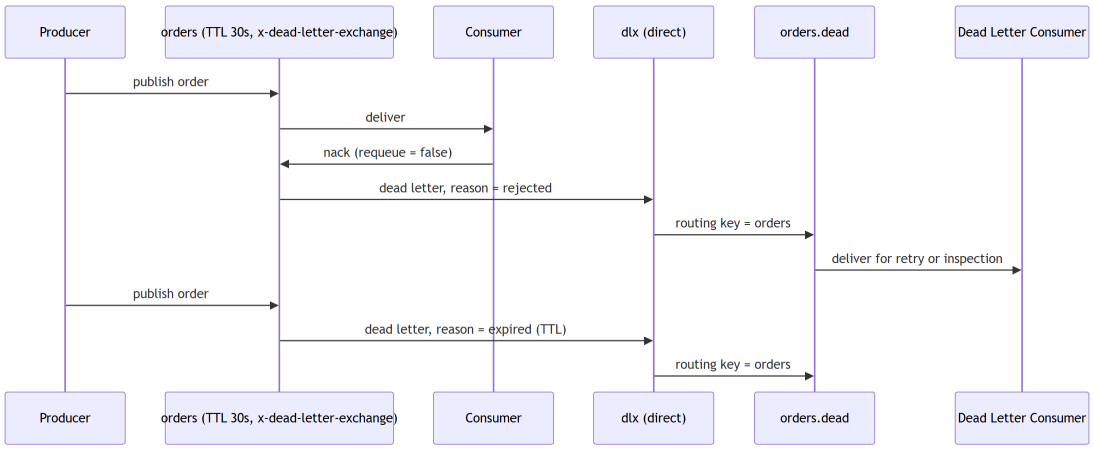
Short description: Messages that expire, are rejected, or overflow a full queue are republished to another exchange instead of being thrown away.

Theory: A queue declared with `x-dead-letter-exchange` forwards any **dead lettered** message to that exchange. A message is dead lettered when it is nacked with `requeue: false`, when its TTL expires, when the queue is full, or when the delivery limit is reached.

How it works: TTL is set either on the queue with `x-message-ttl`, which applies to everything in it, or per message with the `expiration` property. When the timer runs out the broker removes the message and, if a DLX is configured, republishes it there with an `x-death` header recording why and how many times.

Use case example: A retry pipeline. The main queue dead letters failures into a wait queue with a 30 second TTL, whose own DLX points back at the main queue. The message comes back automatically half a minute later, and after a few rounds it lands in a parking queue a human can inspect.

11.1 Architecture diagram



Key Points

- Four causes of dead lettering: rejected, expired, maxlen, delivery limit
- `x-dead-letter-routing-key` overrides the original routing key
- Per message `expiration` and queue `x-message-ttl` both apply, the shorter one wins

Notes

- Only the message at the head of a queue expires, so a slow head delays the rest
- The `x-death` header carries the retry count, use it to stop a loop
- Never point a queue's DLX back at itself without a counter, that is an infinite loop

11.2 Code examples

Dead letter setup (`dlx_setup.js`)

```
const amqp = require('amqplib');

async function setup() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    // The exchange and queue that collect the failures.
    await channel.assertExchange('dlx', 'direct', { durable: true });
    await channel.assertQueue('orders.dead', { durable: true });
    await channel.bindQueue('orders.dead', 'dlx', 'orders');

    // The working queue: 30s TTL, capped length, the rest falls through to the DLX.
    await channel.assertQueue('orders', {
        durable: true,
        arguments: {
            'x-message-ttl': 30000,
            'x-max-length': 10000,
            'x-dead-letter-exchange': 'dlx',
            'x-dead-letter-routing-key': 'orders'
        }
    });

    await channel.prefetch(1);
    channel.consume('orders', (msg) => {
        try {
            handle(JSON.parse(msg.content.toString()));
            channel.ack(msg);
        } catch (err) {
            // requeue: false dead letters it, requeue: true would loop forever.
            channel.nack(msg, false, false);
        }
    }, { noAck: false });
}
setup();
```

Delayed retry with a wait queue (`retry.js`)

```
const amqp = require('amqplib');
const MAX_RETRIES = 3;

async function setup() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    // Wait queue holds the message for 30s, then dead letters it back to the main queue.
    await channel.assertQueue('orders.wait', {
        durable: true,
        arguments: {
            'x-message-ttl': 30000,
            'x-dead-letter-exchange': '',
            'x-dead-letter-routing-key': 'orders'
        }
    });

    channel.consume('orders.dead', (msg) => {
        const deaths = msg.properties.headers['x-death'] || [];
        const count = deaths.length ? deaths[0].count : 0;
        if (count >= MAX_RETRIES) {
            channel.sendToQueue('orders.parked', msg.content, { persistent: true });
        } else {
            channel.sendToQueue('orders.wait', msg.content, { persistent: true });
        }
        channel.ack(msg);
    }, { noAck: false });
}
setup();
```

12. Mirrored and Replicated Queues

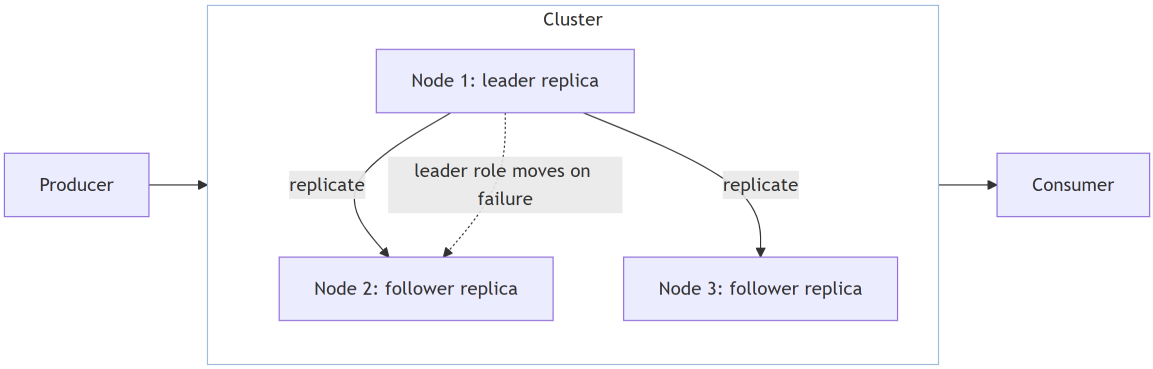
Short description: Keeping copies of a queue on several nodes so the messages survive losing one of them.

Theory: Old **mirrored** classic queues copied a master queue to mirrors using a policy, and are deprecated because a network partition could silently lose acknowledged messages. **Quorum** queues replace them. They use Raft, where one **leader** accepts every write and only confirms it once a **majority** of replicas has it on disk.

How it works: Clients always talk to the leader, no matter which node they connect to, and the broker forwards the traffic. If the leader dies, the remaining replicas elect a new one from those with the most complete log, and clients reconnect to it without losing confirmed messages.

Use case example: A three node cluster running payment events. Node one holds the leader and is taken down for a kernel upgrade. Node two is elected leader, publishers reconnect, and no confirmed payment is lost.

12.1 Architecture diagram



Key Points

- A quorum queue of 3 nodes tolerates 1 failure, 5 nodes tolerate 2
- Use an odd node count, an even one gives no extra safety
- Replication only helps together with publisher confirms and consumer acks

Notes

- Classic queue mirroring policies (`ha-mode`) are removed in RabbitMQ 4
- Set `x-quorum-initial-group-size` to control how many replicas start with
- Replicating every queue is wasteful, replicate only what must not be lost

12.2 Code examples

Declaring a replicated queue (`quorum_queue.js`)

```
const amqp = require('amqplib');

async function setup() {
    // List every node so the client can fail over when one goes down.
    const connection = await amqp.connect([
        'amqp://localhost:5672',
        'amqp://node2:5672',
        'amqp://node3:5672'
    ]);
    const channel = await connection.createConfirmChannel();

    await channel.assertQueue('payments', {
        durable: true,
        arguments: {
            'x-queue-type': 'quorum',
            'x-quorum-initial-group-size': 3,  // leader plus two followers
            'x-delivery-limit': 5
        }
    });

    // The confirm only arrives once a majority of replicas has the message on disk.
    channel.sendToQueue('payments', Buffer.from('payment-42'), { persistent: true },
        (err) => {
            console.log(err ? ' [!] Rejected' : ' [x] Replicated and confirmed');
            connection.close();
        });
}

setup().catch(console.error);
```

Cluster and queue inspection

```
# Cluster health and which nodes are running
rabbitmq-diagnostics cluster_status
rabbitmq-diagnostics check_running

# Queue type, leader node and the current replica set
rabbitmqctl list_queues name type leader members messages

# Move a leader off a node before taking it down for maintenance
rabbitmq-queues quorum_status payments
rabbitmq-upgrade drain

# Grow or shrink the replica set of an existing quorum queue
rabbitmq-queues add_member payments rabbit@node4
rabbitmq-queues delete_member payments rabbit@node2
```

13. Arguments

Short description: Extra options passed when declaring a queue or starting a consumer. They change the behaviour of the broker, unlike properties, which travel with a single message.

Theory: Arguments come in three places.

- **Queue arguments** go in the `arguments` map of `assertQueue` and are fixed for the life of the queue
- **Consumer arguments** go in the `arguments` map of `consume` and affect only that consumer
- **Options** such as `durable` or `persistent` are plain fields, not `x-` arguments

Argument	Example	What it does
`x-queue-type`	`'quorum'`	Picks classic, quorum or stream
`x-message-ttl`	`30000`	Every message expires after 30 seconds
`x-expires`	`1800000`	Delete the queue itself after 30 idle minutes
`x-max-length`	`10000`	Keep at most 10000 messages
`x-max-length-bytes`	`1048576`	Keep at most 1 MB of message bodies
`x-overflow`	`'reject-publish'`	What a full queue does: `drop-head` (default), `reject-publish`, `reject-publish-dlx`
`x-dead-letter-exchange`	`'dlx'`	Where rejected or expired messages are republished
`x-dead-letter-routing-key`	`'orders'`	Routing key used when dead lettering
`x-max-priority`	`10`	Turns it into a priority queue with 10 levels
`x-single-active-consumer`	`true`	Only one consumer receives at a time, the rest wait
`x-delivery-limit`	`5`	Quorum only: dead letter after 5 redeliveries

Notes

- Arguments are fixed at declaration, changing one means deleting the queue or applying a **policy** instead
- An unknown `x-` argument is ignored silently, so a typo simply does nothing
- Values must have the right type, `'30000'` as a string is not the same as `30000`

13.1 Code examples

Consumer arguments

Argument	Example	What it does
`x-priority`	`10`	This consumer is served before lower priority ones
`x-stream-offset`	`'first'`	Where to start reading a stream: `first`, `last`, `next`, an offset or a timestamp
`x-cancel-on-ha-failover`	`true`	Cancel the consumer when the queue moves node

Arguments in practice (`arguments.js`)

```
const amqp = require('amqplib');

async function setup() {
    const connection = await amqp.connect('amqp://localhost');
    const channel = await connection.createChannel();

    // durable is an option, everything under arguments is a broker behaviour.
    await channel.assertQueue('orders', {
        durable: true,
        arguments: {
            'x-queue-type': 'quorum',
            'x-message-ttl': 30000,
            'x-max-length': 10000,
            'x-overflow': 'reject-publish',      // fail the publish instead of dropping
            'x-dead-letter-exchange': 'dlx',
            'x-delivery-limit': 5
        }
    });

    // A backup consumer: it only receives when no higher priority one is connected.
    await channel.consume('orders', (msg) => {
        console.log(' [x] backup handled', msg.content.toString());
        channel.ack(msg);
    }, { noAck: false, arguments: { 'x-priority': 1 } });
}

setup().catch(console.error);
```

14. Quick Reference

Pattern	Exchange	Use it when
Hello World	default (`''`)	One producer, one consumer, no routing
Work Queues	default (`''`)	Spread heavy tasks over a pool of workers
Publish/Subscribe	fanout	Every consumer needs every message
Routing	direct	Consumers pick messages by exact key
Topics	topic	Consumers pick messages by key pattern
RPC	default (`''`)	You need a reply back from the worker
Publisher Confirms	any	Losing a message is unacceptable

Common practices

- Reuse one connection per process and open a channel per task
- Always handle `connection.on('error')` and `'close'`, then reconnect
- Set `prefetch` on every consumer, otherwise one worker hoards the queue
- Acknowledge only after the work is genuinely finished
- Make queues durable and messages persistent when the data matters
- Never block the event loop inside a consumer callback